www.ManzoorTheTrainer.com

Learn Entity Framework Core 5.0 From Scratch

...

Mohd Manzoor Ahmed
(MCT | Founder Of www.ManzoorTheTrainer.com)

MTT

Prerequisite

- Learn MS SQL Server From Scratch
- Learn C#.Net Core From Scratch

What is EF ?

- Entity framework is the latest technology provided by ADO.net for accessing database.
- It is an O/RM tool (Objects Relation Mapping).
- It provides easy and automated way of mapping between Objects(Classes) and Relations(Tables).
- It auto generates most of the data-access code.

Why Use O/RM?

- It provides full intelliSense support for columns names and table names.(Strongly Typed)

```
using (var conn = new SqlConnection("connectionString"))
using (var cmd = new SqlCommand("select * from Employee", conn))
{
    var dt = new DataTable();
    using (var da = new SqlDataAdapter(cmd))
    {
        da.Fill(dt);
    }
}
```

```
foreach (DataRow row in dt.Rows)
{
    int EmpId = Convert.ToInt32(row[0]);
    string EmpName = row["EmpName"].ToString();
}
```

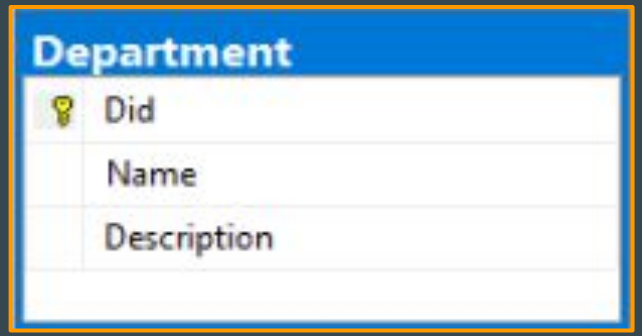
What is EF Core ?

- EF Core is an O\RM tool.
- Entity Framework Core is the latest data access technology for .Net developers.
- EF Core can be used to access NoSQL DB.
- EF Core can run on Windows, Linux, and Apple.
- There are two approaches of working with EF Core
  - **Code-First**
  - Database-First (Reverse Engineering)

Code-First Approach

In the Code-First approach, you focus on your application and start creating classes for it and we call them as business entities or business objects.

```
class Department
{
    public int Did { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
}
```



Getting Started With EF Core

- Start a new .Net Core Console App
- Create an Entity Department

```
class Department
{
    public int Did { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
}
```

EF Core DbContext

- Create a DbContext Class
- DbContext is the top-level object that manages database connection and provides the functionalities on database like Create, Read, Update, and Delete operations.
- Our main data access class using which we perform all the operations should be inherited from DbContext class.
- Install - Nuget Packages
- Microsoft.EntityFrameworkCore.SqlServer

```
class Department
{
    public int Did { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
}
```

```
class OrganizationContext:DbContext
{
}
```

DbSet<TEntity>

- Our OrganizationContext Class Should have DbSet<Department> properties for each entity or object.
- DbSet<TEntity> is used to perform CRUD operations on TEntity.
- Using Departments property, we can perform CRUD operations on Department table with the help of OrganizationContext object.

```
class Department
{
    [Key]
    public int Did { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
}
```

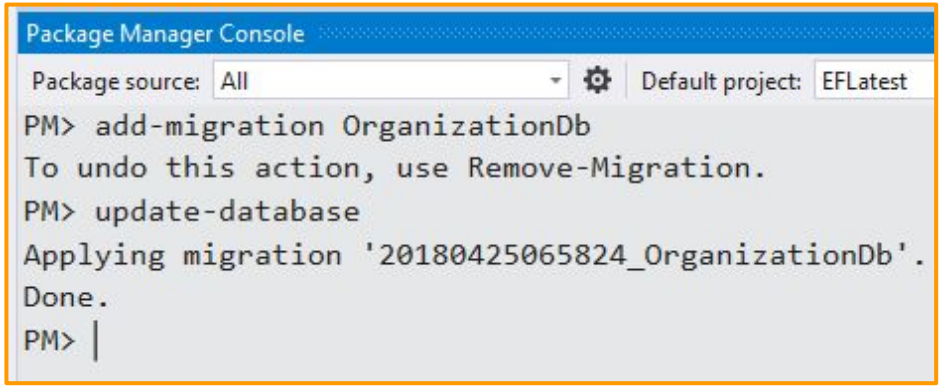
```
class OrganizationContext:DbContext
{
    public DbSet<Department> Departments { get; set; }
}
```

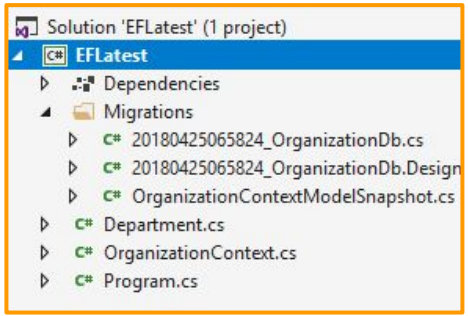
```
OrganizationContext oCntx = new OrganizationContext();
var D = new Department()
{
    Did = 1,
    Name = "Dev",
    Description = "Development"
};
oCntx.Departments.Add(D);
oCntx.SaveChanges();
```

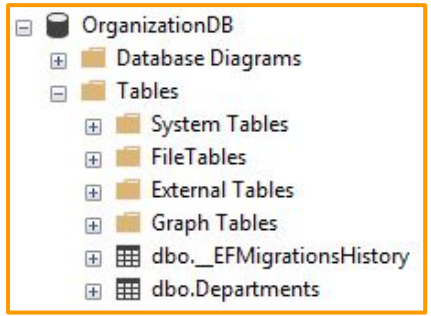
Creating A Database - 5 Steps

1. Install - Nuget Packages Microsoft.EntityFrameworkCore.Tools
2. Override OnConfiguring() method in OrganizationContext with connection string.
3. Open Package Manager Console and run the below commands
4. add-migration OrganizationDb
5. update-database

```
class OrganizationContext:DbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(@"Server=XXXX;Database=OrganizationDB;Trusted_Connection=True;");
    }

    public DbSet<Department> Departments { get; set; }
}
```







Updating Database Structure

- Creating a new migrations. (add-migration <Name>)
- Updating database (update-database)
- Reverting a migration (update-database <Old-Migration-File-Name>)
- Removing a migration (remove-migration) (it will remove the recent migration)
- Generating DbScript (script-migration)
- List Migrations With Status (get-migration)

Data Annotations Attributes

- Table()
- Column()
- Key
- Required
- MaxLength()
- DatabaseGenerated()
- NotMapped


```
[Table("Department")]
class Department
{
    [Key]
    [Column("DepartmentId")]
    public int Did { get; set; }

    [Required]
    public string Name { get; set; }

    [MaxLength(150)]
    public string Description { get; set; }

    [NotMapped]
    public string ShortDescription
    {
        get { return Description.Substring(0, 15); }
    }
    public List<Employee> Employees { get; set; }
}
```

```
class Employee
{
    [Key]
    public int Eid { get; set; }

    public string EName { get; set; }

    [ConcurrencyCheck]
    public double Esalary { get; set; }

    [ForeignKey("Department")]
    public int Did { get; set; }

    public Department Department { get; set; }

    [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
    public DateTime CreatedOn { get; set; }
}
```

Solution To A Given Problem

- Identify Objects
- Identify Relationship among the objects

Entities Design Task - 1 [Organization Management System]

- An organization has multiple number of departments
- An organization has multiple number of employees
- A department can have multiple number of employees
- An employee cannot belongs to multiple departments

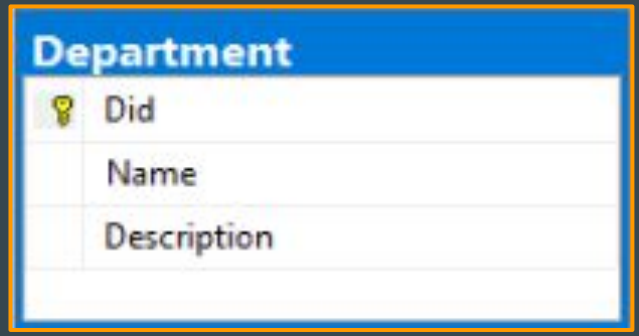
- Objects
  - Department
  - Employee

- Relationship
  - Department - Employee
  - 1 → n
  - n → 1
  - **1 → n (1:M)**

3 Rules For Creating Entities Or Business Objects - #1

- One class for each object → Department.

```
class Department
{
    [Key]
    public int Did { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
}
```

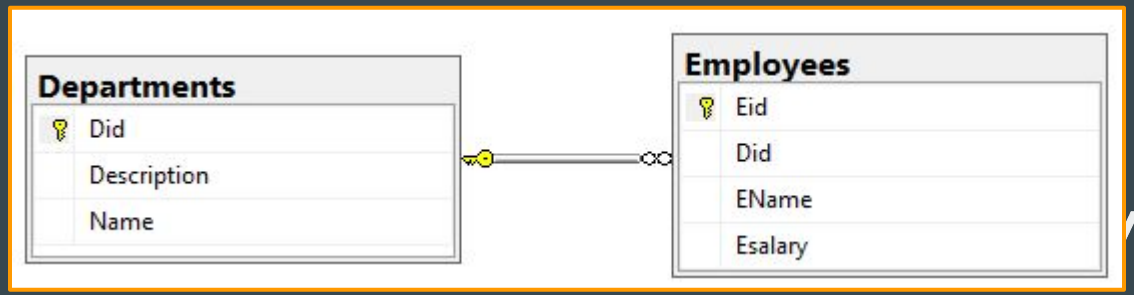


Relationship(1:M) - #2

- Department will have list of employees and Employee will refer to a single department.

```
class Department
{
    [Key]
    public int Did { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public List<Employee> Employees { get; set; }
}
```

```
class Employee
{
    [Key]
    public int Eid { get; set; }
    public string EName { get; set; }
    public double Esalary { get; set; }
    public int Did { get; set; }
    public Department Department { get; set; }
}
```



Entities Design Task - 2 [School Management System]

- A school has multiple number of courses
- A school has multiple number of students
- A student can opt in for multiple number of courses
- A course can be opted by multiple number of students

- Objects
  - Course
  - Student

- Relationship
  - Course - Student
  - 1 → n
  - n → 1
  - **n → n (M:M)**

Relationship(M:M) - Entities - #3

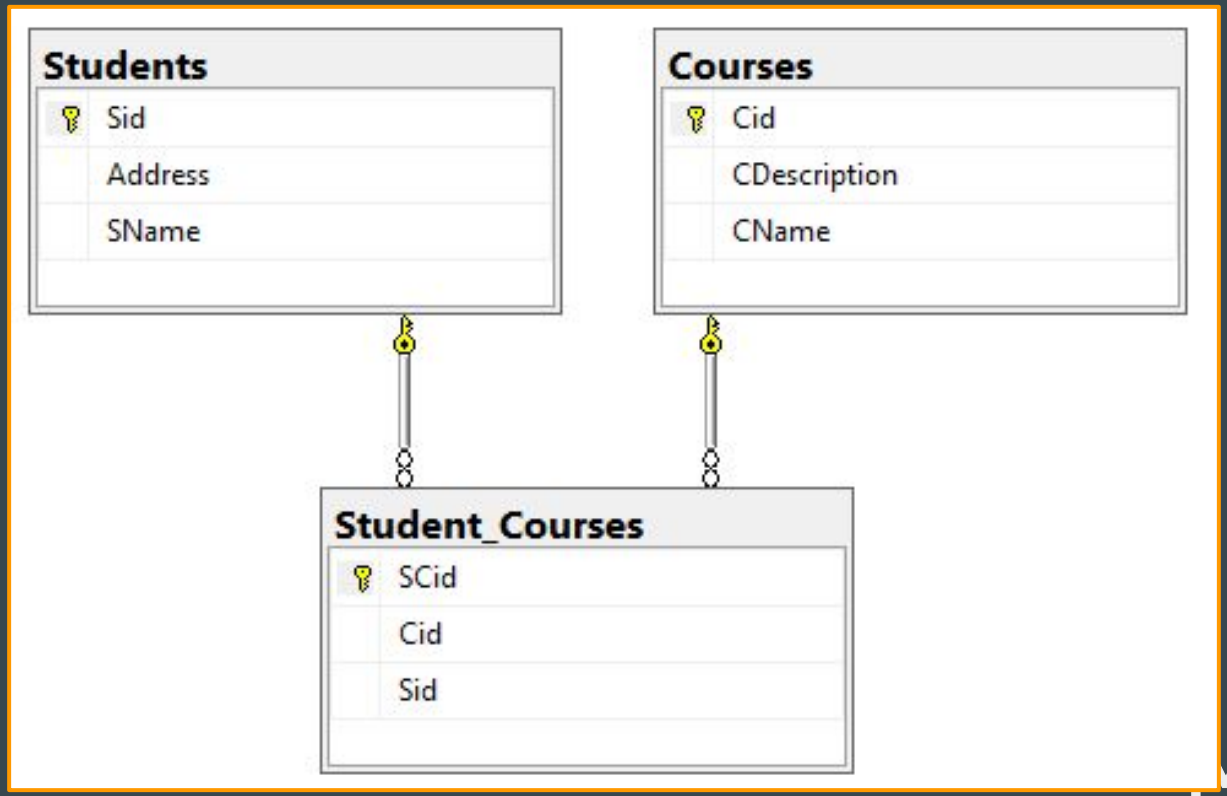
- Student will have list of Student_Course
- Course will also have list of Student_Course
- It will have a new entity Student_Course referring to Student & Course

```
class Student
{
    [Key]
    public int Sid { get; set; }
    public string SName { get; set; }
    public string Address { get; set; }
    public List<Student_Course> Student_Courses { get; set; }
}
```

```
class Course
{
    [Key]
    public int Cid { get; set; }
    public string CName { get; set; }
    public string CDescription { get; set; }
    public List<Student_Course> Student_Courses { get; set; }
}
```

```
class Student_Course
{
    [Key]
    public int SCid { get; set; }
    public int Sid { get; set; }
    public int Cid { get; set; }
    public Student Student { get; set; }
    public Course Course { get; set; }
}
```

Relationship(M:M) - Tables



Entity Design Task - 3 [Stock Management System]

- A Toy manufacturing company manufactures different types of toys.
- The company has several manufacturing plants.
- Each plant manufactures different types of toys.
- A customer can place order for these toys.
- Each customer has multiple ship-to addresses.

Objects And Relationship

- Objects
  - Plant
  - Category
  - Toy
  - Customer
  - ShipToAddress
- Relationship
  - Plant : Category (1→ n)(1:M)
  - Category : Toy (1 → n)(1:M)
  - Customer : ShipToAddress (1 → n)(1:M)
  - ShipToAddress : Toy (n → n) {Order}(M:M)

Entity Design Task - 4 [IT Operation Management System]

Need a web app for IT Operation Management System

- The Employee can raise a request with status open for a new hardware or software component to be installed on their system.
- Employee can also raise a request with status open for update or repair of an existing hardware or software component.
- IT Manager has rights to approve or reject the request. on approval, the request is assigned to IT Assistant.
- IT Assistant can then works on it and completes the task and change the request status to completed.
- Finally IT Manager has rights to close the completed request.

Defining the Roles & Responsibilities

- Employee
  - Can raise a request.
- IT Manager
  - Can approve, reject or closes a request.
  - Can assign it to an IT Assistant on approval.
- IT Assistant
  - Can work on a request assigned to him\her .
  - Can change the status of request to completed.

Identifying the Objects

Need a web app for IT Operation Management System

- The Employee can raise a request with status open for a new hardware or software component to be installed on their system.
- Employee can also raise a request with status open for update or repair of an existing hardware or software component.
- IT Manager has rights to approve or reject the request. on approval, the request is assigned to IT Assistant.
- IT Assistant can then works on it and completes the task and change the request status to completed.
- Finally IT Manager has rights to close the completed request.

Identifying the Objects

Objects

- Employee
- IT Manager
- IT Assistant
- Component
- Request
- Status

Identifying the Objects and Relationships

Object

- Role (M | A | U)
- Employee
- Request
- Component
- Status (OP | AP | RJ | CM | CS)

Relationships

- Role : Employee (1 : M)
- Employee : Component { Request (M :M)}
- Request:Status (1:M)

CRUD - Insert

- Insert

```
OrganizationContext oCntx = new OrganizationContext();
var D = new Department()
{
    Name = "Prod",
    Description = "Production"
};

oCntx.Departments.Add(D);
oCntx.SaveChanges();
//Or
oCntx.Add<Department>(D);
oCntx.SaveChanges();
```

```
OrganizationContext oCntx = new OrganizationContext();
var E = new Employee()
{
    EName = "Manzoor",
    Did = 2,
    Esalary = 5000,
    //It will Ignore this value as it says computed
    CreatedOn = new DateTime(1984, 8, 6)
};

oCntx.Add<Employee>(E);
oCntx.SaveChanges();
```

CRUD - Update

- Update

```
//connected approach
OrganizationContext oCntx = new OrganizationContext();
Department D = oCntx.Departments.Find(2);
D.Description = "MyProduction";
oCntx.SaveChanges();
```

```
//disconnected approach
OrganizationContext oCntx;
public void OnLoad()
{
    oCntx = new OrganizationContext();
}
public void Update(Department D)
{
    oCntx.Update<Department>(D);
    oCntx.SaveChanges();
    //or
    oCntx.Departments.Update(D);
    oCntx.SaveChanges();
}
```

CRUD - Delete

- Delete


```
//connected approach
OrganizationContext oCntx = new OrganizationContext();
Department D = oCntx.Departments.Find(2);
oCntx.Departments.Remove(D);
oCntx.SaveChanges();
```

```
//disconnected approach
OrganizationContext oCntx;
public void OnLoad()
{
    oCntx = new OrganizationContext();
}
public void Delete(Department D)
{
    oCntx.Remove<Department>(D);
    oCntx.SaveChanges();
    //or
    oCntx.Departments.Remove(D);
    oCntx.SaveChanges();
}
```

CRUD - Select

```
OrganizationContext oCntx = new OrganizationContext();

//Get an employee whose Eid=10
Employee E1 = oCntx.Employees.Find(10);

//Get the first employee whose EName=Peter
//even if there are more than one Peter
Employee E2 = oCntx.Employees.Where(x => x.EName == "Peter").FirstOrDefault();

//Get an employee whose EName=Peter and
//throw an exception if more than one Peter is found
Employee E3 = oCntx.Employees.Where(x => x.EName == "Peter").SingleOrDefault();

//Get All the employees
IEnumerable<Employee> Emps = oCntx.Employees.ToList();

//Get All the employees whose Did=1
IEnumerable<Employee> Emps1 = oCntx.Employees.Where(x => x.Did == 1).ToList();

//Get All the employees whose Department Name is Prod
IEnumerable<Employee> Emps2 = oCntx.Employees.Where(x => x.Department.Name == "Prod").ToList();
```

Note : Property in an entity which represents (Primary -Foreign key) relationship with another entity is called as navigation property. As here Department is navigation property in Employees

Immediate Vs Deferred Mode Of Execution

- Immediate Mode:
  - In Immediate mode of query execution query gets executed wherever the query is defined. and it is achieved with the help of linq methods like ToList(), Count() etc.
  - Here query object holds the data which reduces number of hits to SQL server on every request.
  - Return type can be List Or IEnumerable
- Differed Mode:
  - In Differed mode of query execution, Query is executed whenever we use it like, when we access a field or iterating through a List.
  - Here query object hits the server on every request which reduces the size of query objects.
  - Return type can be IQueryable or IEnumerable

Note : IEnumerable is read-only and List is not.

Immediate Vs Deferred Mode Of Execution

```
OrganizationContext oCntx = new OrganizationContext();

//Get All the employees - Immediate mode
IEnumerable<Employee> Emps = oCntx.Employees.ToList();

//It will not hit SQL Server to execute query.
//It just read it from Emps
foreach (var item in Emps)
{
    Console.WriteLine(item.Eid + " " + item.EName);
}

//It will not hit SQL Server to execute query.
//It just read it from Emps
IEnumerable<Employee> EmpsNew = Emps.Where(x => x.Did == 1);
foreach (var item in EmpsNew)
{
    Console.WriteLine(item.Eid + " " + item.EName);
}
```

Immediate Mode Of Execution

```
OrganizationContext oCntx = new OrganizationContext();

//Get All the employees - deferred mode
IEnumerable<Employee> Emps = oCntx.Employees;

//It will now hit SQL Server to execute query.
foreach (var item in Emps)
{
    Console.WriteLine(item.Eid + " " + item.EName);
}

//Deferred mode - It just appends filter to Emps
IEnumerable<Employee> EmpsNew = Emps.Where(x => x.Did == 1);
//It will now hit SQL Server to execute query again.
foreach (var item in EmpsNew)
{
    Console.WriteLine(item.Eid + " " + item.EName);
}
```

Deferred Mode Of Execution

Eager Loading Vs Explicit Loading Vs Lazy Loading

- Eager Loading: If we write a select query on parent table, it not only loads Parent table records but also automatically loads its related table records. It is achieve with the help of Include() method.
- Explicit Loading: If we write a select query on parent table, it loads only parent table records and the child table records are loaded explicitly with the help of Collection() or Reference() methods whenever we need it.
- Lazy Loading: (Microsoft.EntityFrameworkCore.Proxies)If we write a select query on parent table, it loads only parent table records and the child table records are loaded automatically whenever we access the related table with navigation property. (Not Supported In EF Core 2.0 & Support in 2.2)

Eager Loading Vs Explicit Loading Vs Lazy Loading

```
OrganizationContext oCntx = new OrganizationContext();

//Get All departments and their related employees
//Eager Loading
IEnumerable<Department> Depts = oCntx.
    Departments.Include(x=>x.Employees).ToList();

foreach (var D in Depts)
{
    foreach (var E in D.Employees)
    {
        Console.WriteLine(E.Eid + " " + E.EName);
    }
}
```

```
OrganizationContext oCntx = new OrganizationContext();

//Get All the Departments
IEnumerable<Department> Depts = oCntx.Departments.ToList();

foreach (var D in Depts)
{
    //It will now hit SQL Server to execute query.
    //Load All employees of D explicitly  - Explicit Loading
    oCntx.Entry(D).Collection(x=>x.Employees).Load();
    foreach (var E in D.Employees)
    {
        Console.WriteLine(E.Eid + " " + E.EName);
    }
}
```

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlServer(@"Server=DESKTOP-O84LAFN;Database=Organizatio

    //using Microsoft.EntityFrameworkCore.Proxies;
    optionsBuilder.UseLazyLoadingProxies();

}
```

```
OrganizationContext oCntx = new OrganizationContext();

//Get All departments
IEnumerable<Department> Depts = oCntx.Departments.ToList();

foreach (var D in Depts)
{
    //LazyLoading
    //Related employees of D gets loaded automatically
    foreach (var E in D.Employees)
    {
        Console.WriteLine(E.Eid + " " + E.EName);
    }
}
```

Working With Raw SQL in EF Core

- EF Core allows you to work with Raw SQL.
- It is useful if the query you want to perform can't be expressed using LINQ.
- If using a LINQ query is resulting in inefficient SQL being sent to the database.
- Use : FromSql(), ExecuteSqlCommand()
- 3.1.0=>FromSqlRaw(), ExecuteSqlRaw()
- Limitations : https://docs.microsoft.com/en-us/ef/core/querying/raw-sql

```
OrganizationContext oCntx = new OrganizationContext();

//Get All Employees
IEnumerable<Employee> AllEmps = oCntx.Employees
    .FromSql("Select * From Employees")
    .ToList();
```

```
//Get All Employees With Salar >=5000
double Esalary = 6000;
IEnumerable<Employee> SalEmps = oCntx.Employees
   .FromSql("Select * From Employees Where Esalary >= {0}",Esalary)
   .ToList();

//Use SqlParameter class to avoid SQL injection
var paramSalary = new SqlParameter("@Esalary", 6000);
IEnumerable<Employee> SalEmpsWithParam = oCntx.Employees
   .FromSql("Select * From Employees Where Esalary >= @Esalary",paramSalary)
   .ToList();
```

```
//Delete an employee whose Eid is 11 | same approach for insert, update
int NoOfRows = oCntx.Database
    .ExecuteSqlCommand("Delete from Employees where Eid=11");
```

Working With Stored Procedures EF Core

- Create An empty migration file.
- PM> Add-migration All_SPs.
- Create SPs in Up() and drop SPs in Down() of generated migration file.
- Use : FromSqlRaw() to execute an existing stored procedure.
- With return type as an existing entity.
- With Custom return type.

```
public partial class All_SPs : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        var SP1 = @"Create Proc GetAllDepartments
                    as
                    Select * From Department";
        migrationBuilder.Sql(SP1);


        var SP2 = @"Create Proc GetAllEmployeesByDid
                    (@Did as int)
                    as
                    Select * From Employees Where Did=@Did";
        migrationBuilder.Sql(SP2);
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        var SP1 = @"IF OBJECT_ID('GetAllDepartments', 'P') IS NOT NULL
                    DROP PROC  GetAllDepartments";
        migrationBuilder.Sql(SP1);


        var SP2 = @"IF OBJECT_ID('GetAllEmployeesByDid', 'P') IS NOT NULL
                    Drop Proc GetAllEmployeesByDid";
        migrationBuilder.Sql(SP2);
    }
}
```

```
OrganizationContext oCntx = new OrganizationContext();
IEnumerable<Department> Depts = oCntx.Departments
                                .FromSql("GetAllDepartments")
                                .ToList();
```

```
//With input parameter
OrganizationContext oCntx = new OrganizationContext();
IEnumerable<Employee> Depts = oCntx.Employees
                              .FromSql("GetAllEmployeesByDid 1")
                              .ToList();
```

Default Transaction In EF Core

- Say we are inserting 1 record and updating 2 records and finally calling SaveChanges().
- By default, SaveChanges() are applied in a transaction.
- Which means If any of the operations fail, then the transaction is rolled back.
- You can also manually control transactions if you have multiple SaveChanges() method Call.

AcountId	Balance	Name
1	34000	Peter
2	5000	Mark
3	69000	Lilly
NULL	NULL	NULL

```
BankContext bankContext = new BankContext();

bankContext.Add<Account>(new Account() { Name = "Adam", Balance = 7500 });

var accout1 = bankContext.Accounts.Find(2);
accout1.Balance = accout1.Balance - 5000;

var accout2 = bankContext.Accounts.Find(3);
//An exception will be thrown here
accout2.Balance = accout2.Balance + double.Parse("500f");

//This method will not be invoked. Which will leave DB untuched
bankContext.SaveChanges();
```

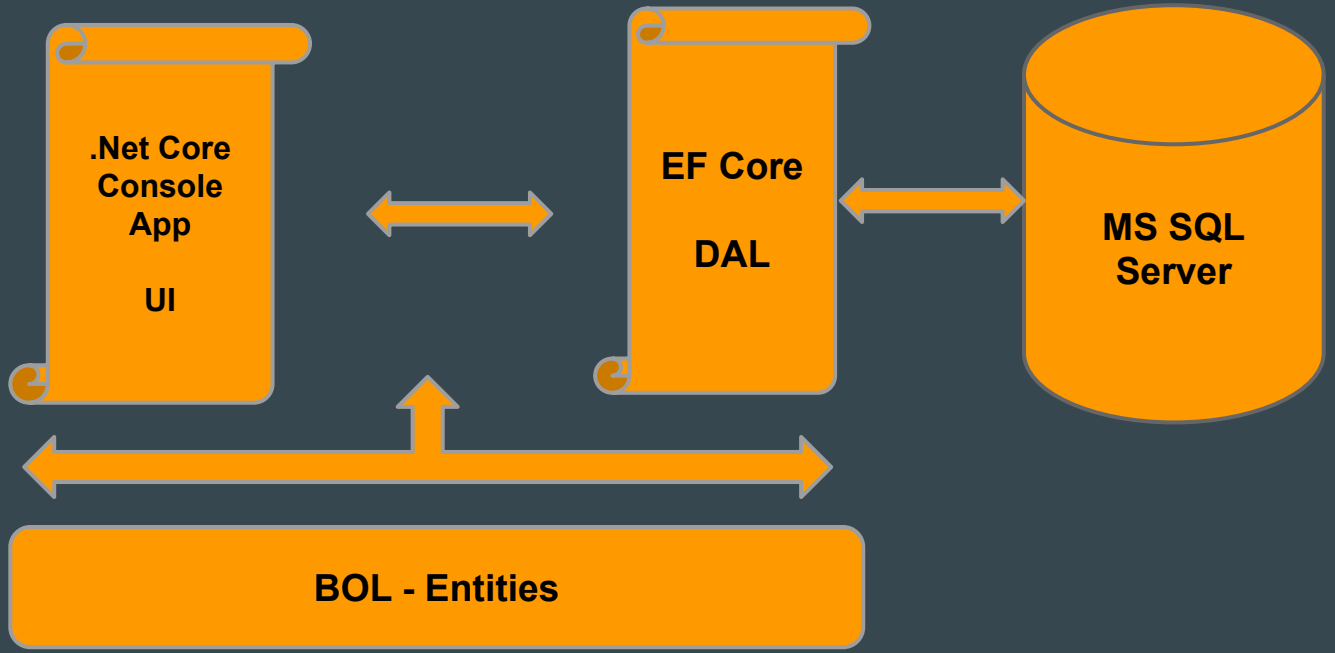
AcountId	Balance	Name
1	34000	Peter
2	5000	Mark
3	69000	Lilly
NULL	NULL	NULL

Manually Controlling Transaction In EF Core

- You can also manually control transactions if you have multiple SaveChanges() method Call.
- We use the DbContext.Database to begin, commit, and rollback transactions.
- Transactions are always applied on try catch blocks

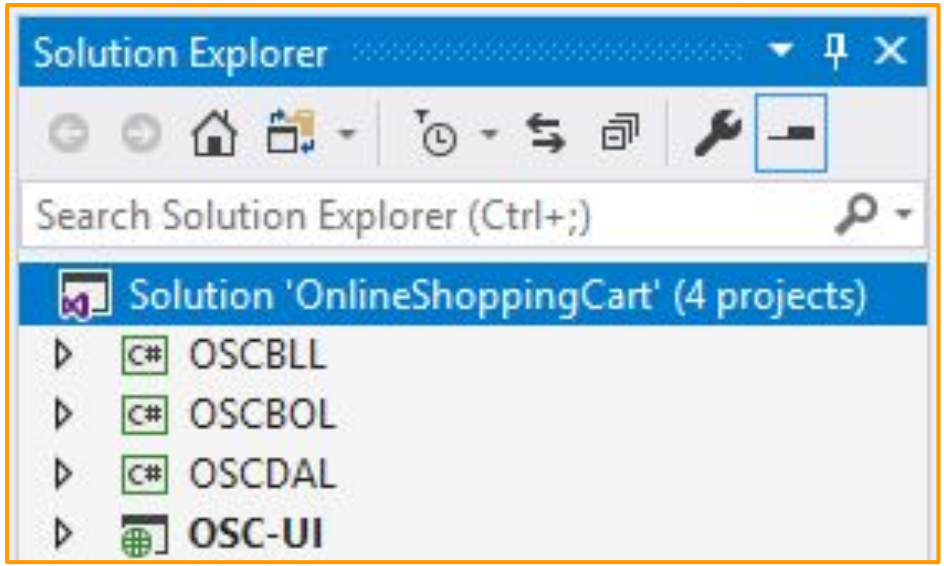
```
BankContext bankContext = new BankContext();
using (var transaction = bankContext.Database.BeginTransaction())
{
    try
    {
        bankContext.Add<Account>(new Account() { Name = "Adam", Balance = 7500 });
        bankContext.SaveChanges();

        var accout1 = bankContext.Accounts.Find(2);
        accout1.Balance = accout1.Balance - 5000;
        bankContext.SaveChanges();

        var accout2 = bankContext.Accounts.Find(3);
        accout2.Balance = accout2.Balance + double.Parse("500f");
        bankContext.SaveChanges();

        transaction.Commit();
    }
    catch (Exception E)
    {
        transaction.Rollback();
        Console.WriteLine(E.Message);
    }
}
```

n - Layered Approach - .Net Core Library



3 - Layered Approach

1. Add BOL
2. For Attributes Install Package :
   a. System.ComponentModel.Annotations
3. Add DAL as EF Core Install Packages :
   a. Microsoft.EntityFrameworkCore.SqlServer
   b. Microsoft.EntityFrameworkCore.Tools
4. add-migration & update-database
5. Write data access classes with repository pattern.
6. Add BLL & Write business classes
7. Add .Net Core Console UI
8. Install Package In UI :
   a. Microsoft.EntityFrameworkCore.SqlServer



https://www.learnentityframeworkcore.com/configuration/fluent-api

Thank You